

PAL Cheatsheet

import PythonAutomataLibrary as pal

Default: safe mode. Call `pal.FastMode()` **before** constructing any PAL object to opt into fast mode. Compile model functions with `@pal.njit`.

Create state

Need	Constructor
Typed lattice	<code>pal.NewGrid((nx, ny), dtype)</code>
Individual agents	<code>pal.NewAgentGrid((nx, ny), numAgentProps=0, isStackable=False)</code>
Population counts	<code>pal.NewPopGrid((nx, ny), capacity=None)</code>
Continuous field	<code>pal.NewPDEgrid((nx, ny))</code>
Integer list	<code>pal.NewIList()</code>
Random count sampler	<code>pal.NewMultinomial()</code>

Dimensions: 1-3 axes; negative size wraps that axis (`(-nx, ny)`). `pal.NewAgentGrid()` supports nonspatial populations.

Shared geometry

Task	Syntax
Size / wrap	<code>grid.xDim, grid.yDim, grid.nDims, grid.wrapX</code>
Coordinates -> index	<code>grid.ToI(x, y)</code>
Index -> coordinates	<code>grid.ItoX(i), grid.ItoY(i)</code>
Rectangular region	<code>grid.Box(x1, x2, y1, y2)</code> (half-open)
Neighbor coordinates	<code>grid.Hood(hood, x, y)</code>
Neighborhood offsets	<code>pal.MooreHood(2, True), pal.VonNeumannHood(2), pal.CircleHood(2, 3)</code>

Grid

```
g = pal.NewGrid((40, 40), float)
x, y = 10, 12
g[x, y] = 1.0
value = g[x, y]
copy = g[2:5, 3:8]    # detached NumPy array
```

AgentGrid

```
agents = pal.NewAgentGrid((40, 40), numAgentProps=2)
a = agents.NewAgentSQ(10, 12) # lattice site (10,12); continuous center (10.5,12.5)
agents[a, 0] = 1.5           # property
agents.MoveSQ(a, 11, 12)
for a in agents.All():       # snapshot; safe to Dispose during loop
    agents.Dispose(a)
```

`agents.GetPop()`, `agents.Alive(a)`, `agents.AgentsAt(x, y)`, `agents.LastAgent(x, y)`, `agents.counts[x, y]`. Lattice position: `I(a)`, `XSQ(a)`, `YSQ(a)`. Continuous position: `NewAgent(x, y)`, `Move(a, x, y)`, `X(a)`, `Y(a)`. Wrapped displacement: `DispWrapX(x1, x2)`, `DispWrapY(y1, y2)`.

PopGrid and PDEgrid

```
pop = pal.NewPopGrid((40, 40))
x, y = 10, 12
pop.Add(1, x, y) # pending
pop.Update()     # apply simultaneously
pop.GetPop()     # total
pop.Reset()      # clear current + pending

field = pal.NewPDEgrid((40, 40))
dt, dx, dy = 0.01, 1.0, 1.0
field.SetTimeSpaceStep(dt, dx, dy)
field.Diffusion(0.1)
field.Update()
```

Direct `pop[x, y] = n / field[x, y]` = value changes current state immediately. Add changes only pending state until `Update`. `PopGrid(..., capacity=n)` caps each site at `n`, not the total population.

PDE methods: `Diffusion`, `DiffusionMask`, `DiffusionField`, `DiffusionInterfaces`, `DiffusionADI`, `Advection`, `AdvectionField`, `AdvectionInterfaces`. Radial 1D grids: `DiffusionRadialCircle`, `DiffusionRadialSphere`.

Lists and randomness

```
items = pal.NewIList()
items.Append(4).Append(8)
for i in items.Iter(): pass # live/no copy
snapshot = items.All()      # detached copy
items.Shuffle()
pal.Seed(123)
u = pal.Random()
i = pal.RandInt(10)         # 0..9
m = pal.NewMultinomial()
count = m.Binomial(20, 0.3)
```

Draw

```
pix, window = pal.StartPixWindow(40, 40, scale=4)
x, y = 10, 12
pix[x, y] = 0xFF0000
window.Update()
window.Save("frame.png", block=True)
window.Close()
```

`pal.StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False)` returns `(pix, window)`. Use `window.Save(path, block=True)` to wait for an image file. For unattended output pass `headless=True`; the image example above can then run without a display. Use `window.StartGif(path, delay=100)`, `window.AddGifFrame(block=False)`, and `window.StopGif()` for animation; call `Update()` before capturing a frame. Always call `Close()` when finished.

For 2D/3D geometry use `pal.StartOpenGLWindow(...)` with `Circle`, `Box`, `BoxSQ`, `Line`, `Borders`, `Camera`, `Background`, and `Clear`. Headless OpenGL requires a supported rendering backend.

More detail: [Manual](#) / [API Guide](#) / [API Reference](#).